David Jimenez Acevedo

Fecha: 15/10/2026

Entrega final

Enlace al video de YouTube: https://youtu.be/VWVipfx0Nq8

Aviso: Todos los datos son ficticios

Triage automatizado de tickets de soporte interno

Caso de uso: mesa de ayuda de TI y mantenimiento en una planta manufacturera

1. Mapa de arquitectura

El sistema tiene dos ramas dentro de un mismo flujo de n8n, más un workflow independiente para errores. La primera rama es reactiva, se dispara con cada correo entrante; la segunda es programada y produce el panel de control. Cada rama escribe su estado en la misma base de Airtable.

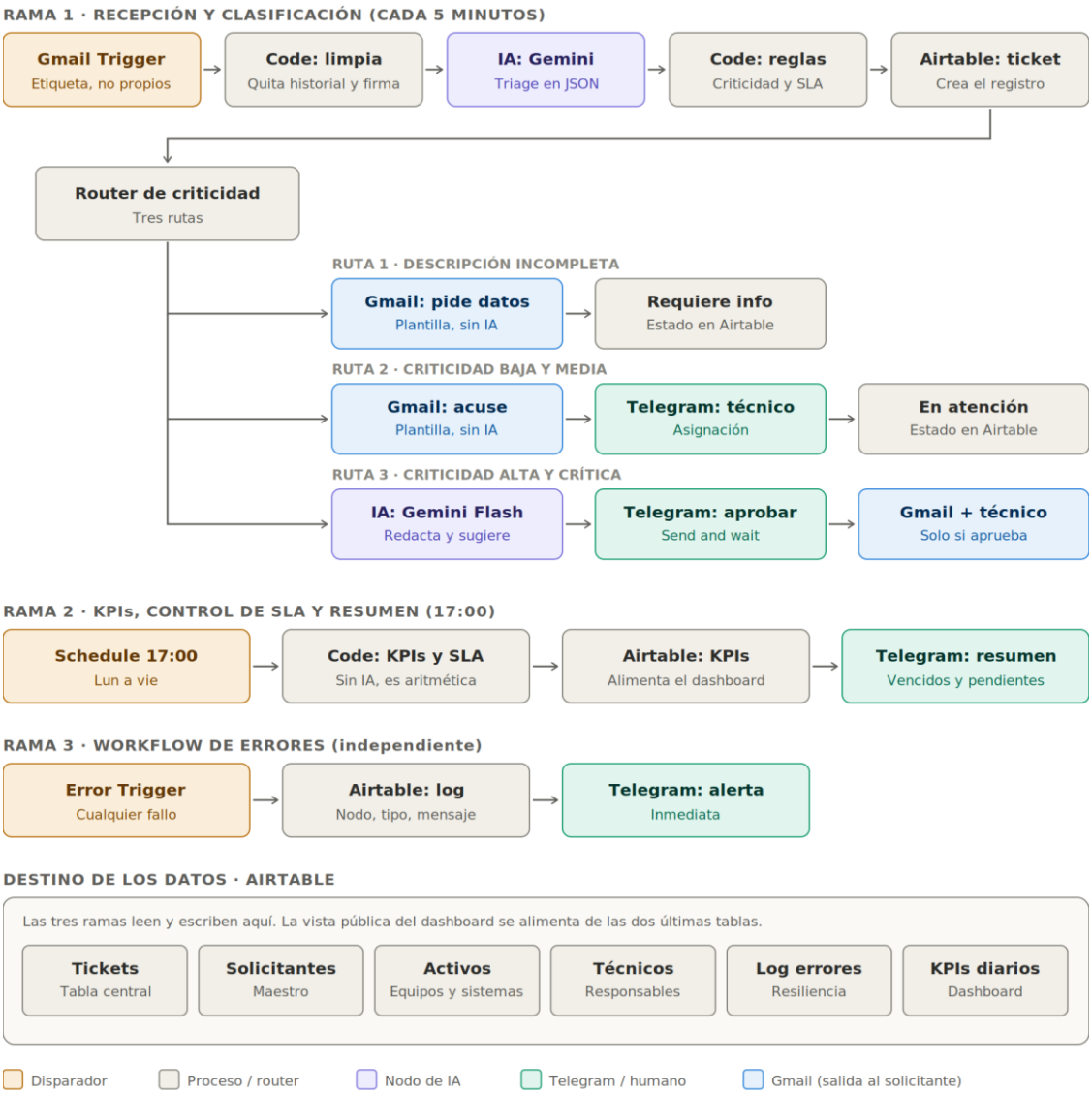


Figura 1. Mapa completo del sistema: disparadores, routers, nodos de IA, APIs de salida y destino de los datos.

1.1 Tiempos de atención comprometidos

El tiempo de primera atención depende de la criticidad final y es un número de horas, no una etiqueta. Esto importa porque el control de incumplimientos compara horas transcurridas contra horas comprometidas, número contra número.

Criticidad	Horas	Definición operativa
Crítica	2	Línea de producción detenida, sistema caído para toda la planta, riesgo de inocuidad o seguridad
Alta	4	Equipo o sistema operando con falla parcial, o un área completa afectada
Media	8	Un usuario o equipo no esencial afectado, con alternativa de

		trabajo disponible
Baja	24	Solicitud de mejora, duda, instalación programada

2. Estructuras de datos

Seis tablas: una central de tickets, tres maestros que le dan contexto y dos de control. La tabla de tickets es la única que el flujo escribe de forma intensiva; los maestros se mantienen a mano y cambian poco.

2.1 Modelo relacional

Tickets enlaza con Solicitantes, Activos y Técnicos; Activos enlaza con Técnicos. Ningún dato queda aislado.

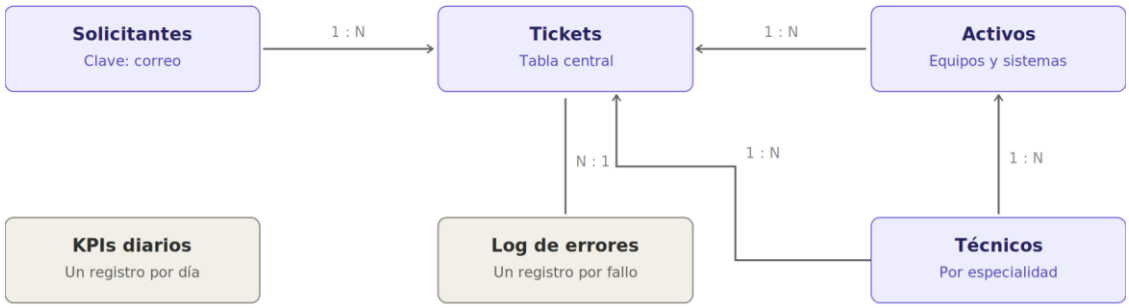


Figura 2. Tablas vinculadas. Un técnico puede ser responsable de varios activos y, a la vez, estar asignado a varios tickets.

2.2 Esquema de tablas

Los campos de estado del sistema son Estado, que recorre Recibido → Requiere info / En atención / Pendiente aprobación → Atendido / Gestión manual / Cerrado / Error, y Decisión humana, con Pendiente, Aprobado, Rechazado o No aplica.

Tickets		
Campo	**Tipo**	**Descripción**
ID ticket	Texto (primario)	TK-AAAAMMDD-HHMMSS. Lo genera el flujo, es único y sirve de referencia en los correos.
Fecha recepción	Fecha y hora	Momento en que entró el correo.
Solicitante	Vínculo	Hacia Solicitantes, resuelto por el correo del remitente.
Correo solicitante	Email	Se guarda también en texto, para no perder la trazabilidad si el remitente no está en el maestro.

Asunto	Texto	Asunto original del correo.
Descripción limpia	Texto largo	Cuerpo sin historial citado ni firma. Es lo único que ve el modelo.
Categoría	Selección	Ocho valores del catálogo, cuatro de TI y cuatro de mantenimiento.
Criticidad IA	Selección	Lo que propuso el modelo. Se conserva para poder auditar su desempeño.
Criticidad final	Selección	Después de aplicar las reglas de negocio. Es la que manda.
SLA horas	Número	2, 4, 8 o 24 según la criticidad final.
Resumen IA	Texto	Una línea, para listados y notificaciones.
Acción sugerida IA	Texto largo	Primeras acciones propuestas. Solo se llena en la ruta de criticidad alta.
Activo	Vínculo	Hacia Activos, cuando el modelo logró identificarlo.
Técnico asignado	Vínculo	Responsable del activo o, en su defecto, de la especialidad.
Confianza IA	Selección	Alta, media o baja. La baja fuerza revisión humana.
Información suficiente	Checkbox	Si está en falso, el flujo pide datos en lugar de diagnosticar.
Estado	Selección	Recibido, Requiere info, En atención, Pendiente aprobación, Atendido, Gestión manual, Cerrado, Error.
threadId, messageId	Texto	Hilo y mensaje de Gmail. Permiten responder en la misma conversación y detectar correos repetidos del mismo hilo.
Decisión humana	Selección	Pendiente, Aprobado, Rechazado, No aplica.
Fecha primera respuesta	Fecha y hora	Momento en que salió la respuesta al solicitante.
Horas de respuesta	Fórmula	DATETIME_DIFF({Fecha primera respuesta}, {Fecha recepción}, 'hours')
Cumple SLA	Fórmula	IF({Horas de respuesta} <= {SLA horas}, "Sí", "No"). Número contra número.

Maestros y control	
Tabla	**Campos**
Solicitantes	Correo (primario), Nombre, Área, Sede, Activo.
Activos	Código (primario), Nombre, Tipo (Equipo o Sistema), Área, Crítico para producción (checkbox), Técnico responsable (vínculo), Palabras clave.
Técnicos	Nombre (primario), Correo, Especialidad (TI, Mecánico, Eléctrico,

	Instrumentación), Chat ID de Telegram, Activo.
Log de errores	Fecha y hora, Rama, Nodo, Tipo (API IA, Gmail, Airtable, Datos), Mensaje, Ticket (vínculo), Resuelto.
KPIs diarios	Fecha (primario), Recibidos, Clasificados, Requieren info, Criticidad alta, Aprobados, Rechazados, Cumplen SLA, Incumplen SLA, Errores, Tasa de cumplimiento (fórmula), Tasa de error (fórmula).

El campo **Palabras clave** de Activos es el que permite que el modelo relacione «la envasadora 2» o «la empacadora de la línea 2» con el código `ENV-002`. Esa lista se envía en el prompt, por lo que el catálogo de activos debe mantenerse corto y bien nombrado.

2.3 Esquemas JSON de transferencia

Cada integración tiene un contrato de datos explícito. Documentarlos permite cambiar un nodo sin romper el siguiente, y hace depurable el flujo cuando algo falla.

T1. Gmail Trigger → nodo Code de limpieza

```
{
  "id": "19b2f1c4d8e7a6b5",
  "threadId": "19b2f1c4d8e7a6b5",
  "subject": "problema con la envasadora",
  "from": { "value": [{ "address": "operario1@empresa-ficticia.com",
"name": "Juan Perez" }] },
  "text": "Buenos dias, la envasadora 2 esta botando producto...\n\n>
mensaje anterior citado"
}
```

T2. Code de limpieza → nodo de IA

```
{
  "ticket_id": "TK-20261003-084512",
  "correo": "operario1@empresa-ficticia.com",
  "nombre": "Juan Perez",
  "area": "Produccion",
  "asunto": "problema con la envasadora",
  "descripcion_limpia": "Buenos dias, la envasadora 2 esta botando
producto y tuvimos que
                        parar la linea hace 20 minutos.",
  "threadId": "19b2f1c4d8e7a6b5",
  "messageId": "19b2f1c4d8e7a6b5"
}
```

T3. Nodo de IA → n8n (salida estructurada obligatoria)

```
{
  "categoria": "Mantenimiento mecanico",
  "criticidad_propuesta": "alta",
```

```
  "resumen": "Envasadora 2 descarga producto fuera de la bolsa",
  "activo_detectado": "ENV-002",
  "parada_de_linea": true,
  "informacion_suficiente": true,
  "datos_faltantes": [],
  "confianza": "alta"
}
```

El modelo responde solo con JSON, sin texto envolvente ni bloques de código. Si no logra encajar la solicitud en el catálogo, devuelve confianza baja, y el flujo la trata como revisión humana en lugar de forzar una categoría.

T4. Code de reglas → Airtable (creación del ticket)

```
{
  "ID ticket": "TK-20261003-084512",
  "Fecha recepcion": "2026-10-03T08:45:12-05:00",
  "Correo solicitante": "operario1@empresa-ficticia.com",
  "Asunto": "problema con la envasadora",
  "Descripcion limpia": "...",
  "Categoria": "Mantenimiento mecanico",
  "Criticidad IA": "Alta",
  "Criticidad final": "Critica",
  "SLA horas": 2,
  "Resumen IA": "Envasadora 2 descarga producto fuera de la bolsa",
  "Confianza IA": "Alta",
  "Informacion suficiente": true,
  "Estado": "Recibido",
  "threadId": "19b2f1c4d8e7a6b5",
  "messageId": "19b2f1c4d8e7a6b5",
  "ruta": "alta"
}
```

La criticidad subió de Alta a Crítica porque el activo `ENV-002` está marcado como crítico para producción y el modelo reportó parada de línea. Esa decisión la toma el código, no el modelo, y queda trazable al conservar ambos valores.

T5. n8n → Telegram (aprobación) y respuesta del humano

```
Salida hacia Telegram:
{
  "chat_id": "{{ $env.TG_CHAT_ID }}",
  "mensaje": "TK-20261003-084512 | CRITICA | Mantenimiento
mecanico\nEnvasadora 2 descarga
              producto fuera de la bolsa\nSolicitante: Juan Perez
(Produccion)\nSLA: 2 h\n\n
              Borrador de respuesta:\n[texto generado]",
  "approvalOptions": { "approvalType": "double", "labels": ["Aprobar",
"Rechazar"] }
}
```

```
Retorno del nodo Send and Wait:
{ "data": { "approved": true } }
```

T6. Code de KPIs → Airtable (KPIs diarios)

```
{
  "Fecha": "2026-10-03",
  "Recibidos": 18, "Clasificados": 17, "Requieren info": 3,
  "Criticidad alta": 4, "Aprobados": 3, "Rechazados": 1,
  "Cumplen SLA": 15, "Incumplen SLA": 2, "Errores": 1
}
```

3. Optimización de costos

El criterio no es usar el modelo más barato, sino ubicar cada tarea donde el costo deja de ser el factor limitante y empieza a serlo la confiabilidad. En este flujo hay cinco tareas, y tres de ellas no necesitan IA.

3.1 Tarifas vigentes

Modelo	Entrada (USD / 1M tokens)	Salida (USD / 1M tokens)	Perfil
Gemini 3.1 Flash-Lite	0,25	1,50	Clasificación, extracción, enrutamiento
Gemini 3.5 Flash-Lite	0,30	2,50	Igual perfil, generación intermedia
Gemini 3.8 Flash	0,75	3,75	Redacción de propósito general
Gemini 3.5 Flash	1,50	9,00	Redacción con más exigencia
Gemini 3.1 Pro	2,00	12,00	Razonamiento largo (4,00 / 18,00 sobre 200K tokens)
Gemini 2.5 Flash-Lite	0,10	0,40	Más barato, pero se retira el 16 de octubre de 2026

Tarifas estándar consultadas el 3 de octubre de 2026 en la documentación de la Gemini API. La API de lotes descuenta el 50 % de entrada y salida; el context caching reduce hasta un 90 % el costo de la entrada repetida.

3.2 Matriz de decisión por tarea

Tarea	Volumen mensual	Exigencia	Latencia	Costo de equivocarse	Decisión
Triage: categoría, criticidad,	396	Media: texto libre a JSON con	Minutos	Medio: una mala categoría	**Gemini 3.1 Flash-Lite.** La tarea es clasificación y

Tarea	Volumen mensual	Exigencia	Latencia	Costo de equivocarse	Decisión
activo		catálogo cerrado		desvía el ticket, pero las reglas de negocio corrigen la criticidad	extracción, exactamente su tier, y concentra el 83 % de las llamadas. Se usa con `responseMimeType: application/json`, que obliga al modelo a devolver JSON válido.
Redactar respuesta a ticket crítico	79	Alta: tono institucional y acciones iniciales	Minutos	Bajo: pasa por aprobación humana antes de salir	**Gemini 3.8 Flash.** Es el texto que lee una persona en un momento de presión. Al ser el 20 % del volumen, subir de gama aquí cuesta centavos.
Acuse de recibo de criticidad baja y media	317	Nula: texto fijo con variables	Segundos	Bajo	**Sin IA.** Plantilla con número de ticket, categoría y SLA. Un modelo aquí solo agrega costo y riesgo de inventar datos.
Pedir datos faltantes	79	Nula: la lista de faltantes ya viene del triage	Segundos	Bajo	**Sin IA.** Plantilla que enumera los campos que el modelo marcó como ausentes.
KPIs y resumen diario	22	Nula: es aritmética	Segundos	Medio: alimenta el dashboard	**Sin IA.** Un nodo Code da el resultado exacto siempre; un modelo puede equivocarse al contar.
Análisis mensual de	1 lote de 396	Media	Horas	Bajo	**Gemini 3.1 Flash-Lite vía Batch API.**

Tarea	Volumen mensual	Exigencia	Latencia	Costo de equivocarse	Decisión
fallas recurrentes					Nadie espera el resultado, así que el 50 % de descuento sale gratis: USD 0,14 en vez de USD 0,29 por corrida.

3.3 Costo estimado y ahorro

Supuestos: 22 días hábiles al mes, 18 tickets diarios (396 al mes), 20 % de criticidad alta o crítica (79), 20 % con información insuficiente (79). Tokens por llamada: triage 1.100 de entrada y 250 de salida; redacción 1.300 de entrada y 450 de salida. En ambos nodos se fija `thinkingBudget: 0`, porque los tokens de razonamiento se facturan como salida y ninguna de las dos tareas los necesita.

Escenario	Costo mensual	Costo anual	Comentario
A. Diseño implementado Flash-Lite para triage, Flash para redactar, plantillas para lo demás	**USD 0,47**	**USD 5,64**	Triage USD 0,26 y redacción USD 0,21.
B. Un solo modelo de gama alta para todo (Gemini 3.1 Pro)	USD 2,69	USD 32,28	Casi seis veces más caro sin ganancia de calidad: ninguna de las dos tareas la necesita.
C. El diseño, pero usando IA también en acuses, solicitudes de datos y resumen	USD 0,71	USD 8,52	Pagar por generar texto que ya es fijo, y perder el determinismo.

Ahorro. Frente al escenario B, el diseño ahorra USD 26,64 al año (82 %). Frente al escenario C, ahorra USD 2,88 al año (34 %). A este volumen las cifras absolutas son pequeñas; el punto es que las decisiones escalan linealmente: si la mesa de ayuda pasa a 180 tickets diarios, el ahorro anual frente a B sube de USD 27 a USD 266 sin cambiar una línea del flujo.

3.4 Por qué no el modelo más barato de todos

Gemini 2.5 Flash-Lite cuesta 0,10 / 0,40, menos de la mitad que el modelo elegido, y haría el triage por unos USD 0,08 mensuales. No se usa por una razón que no es técnica: Google lo retira el 16 de octubre de 2026. Montar el flujo sobre un modelo con fecha de apagado a dos semanas significa volver a probar prompts y validar salidas justo cuando el sistema ya esté en producción. El piso real del catálogo pasa a ser Gemini 3.1 Flash-Lite, y ese es el que conviene adoptar desde el inicio.

La misma lógica aplica al comparar proveedores. GPT-4o mini, a 0,15 / 0,60, saldría algo más barato que Gemini 3.1 Flash-Lite, con una diferencia de unos USD 2 al año a este volumen. No compensa cambiar de proveedor por ese monto cuando el resto del ecosistema (correo, calendario, documentos) ya es Google y la credencial se administra en el mismo lugar.

Ambas decisiones se revisan si el volumen se multiplica por veinte. En ese punto las diferencias pasan a tres cifras anuales y vale la pena medir la precisión de los modelos sobre un conjunto de tickets reales ya clasificados a mano.

3.5 Palancas de costo ya incorporadas

Palanca	Aplicación en el flujo
Razonamiento desactivado	thinkingConfig.thinkingBudget = 0 en el nodo de triage. Los modelos Gemini 3.x facturan los tokens de razonamiento como salida, que cuesta seis veces más que la entrada. Clasificar contra un catálogo cerrado no requiere razonamiento extendido.
Salida estructurada	responseMimeType: application/json obliga al modelo a devolver JSON válido. Evita gastar tokens en texto envolvente y elimina casi por completo los errores de parseo.
Límite de tokens de salida	350 en el triage y 600 en la redacción.
Limpieza de la entrada	Se elimina el historial citado y la firma antes de llamar al modelo. En un correo reenviado tres veces, esto reduce la entrada a menos de la mitad.
Disparador específico	El Gmail Trigger filtra por etiqueta. Sin ese filtro, cada correo del buzón consumiría una ejecución y una llamada al modelo.
Un ticket por hilo	Si el hilo ya tiene ticket, la ejecución termina antes de llamar al modelo. Las respuestas del solicitante en la misma conversación no generan llamadas nuevas.
Catálogo acotado	Se envían ocho categorías y el listado corto de activos, no toda la base. El prompt se mantiene por debajo de 1.200 tokens.
Batch API	Reservada para el análisis mensual agregado, con 50 % de descuento.
Context caching	El catálogo de categorías y activos es idéntico en cada llamada. A volúmenes mayores, cachearlo reduce hasta un 90 % esa porción de la entrada, aunque cobra almacenamiento por hora: solo conviene por encima de unas 50 llamadas diarias.

4. Seguridad y resiliencia

4.1 Minimización de datos

Principio	Implementación

Principio	Implementación
El modelo recibe lo mínimo	Al nodo de IA llegan el texto limpio del problema, el catálogo de categorías y el listado de activos. No se envía el nombre completo del solicitante, su correo, su cargo ni el historial de tickets anteriores: nada de eso aporta a clasificar una falla.
Sin historial ni firmas	El texto se corta en el primer marcador de cita y se eliminan los bloques de firma, que suelen contener celulares, cargos y avisos de confidencialidad.
Maestros acotados	De cada solicitante se almacenan correo corporativo, nombre, área y sede. No hay datos personales adicionales.
Chat IDs protegidos	Los identificadores de Telegram de los técnicos viven en Airtable y no se exponen en la vista pública ni en el JSON del flujo.
Credenciales fuera del flujo	Todas las llaves viven en el gestor de credenciales de n8n o en variables de entorno. Ningún nodo Code contiene tokens, y el JSON exportado al repositorio no los incluye.
Repositorio con datos ficticios	Solicitantes, activos, técnicos y tickets del repositorio son inventados. Las capturas se toman sobre esa base.
Vista pública agregada	El dashboard compartido expone conteos y tasas por día, no la descripción de los tickets ni la identidad de los solicitantes.

4.2 Rutas de error

n8n implementa el equivalente a las directivas Resume y Break mediante dos mecanismos: la opción `On Error` de cada nodo, que permite continuar por una salida de error en lugar de detener la ejecución, y el *Error Workflow*, que se dispara ante cualquier fallo no capturado.

Punto de fallo	Configuración	Comportamiento
API de IA no responde	On Error: continuar por salida de error, 2 reintentos con 5 s de espera	El ticket se crea igual, con estado Error y sin clasificar, se registra en el log y llega alerta por Telegram. Nunca se pierde una solicitud por una falla de la API.
El modelo devuelve JSON inválido	Validación en nodo Code con try / catch	El ticket queda con confianza baja y estado Gestión manual. No se envía respuesta automática.
Categoría fuera del catálogo	Validación contra la lista permitida	Se reemplaza por «Sin clasificar» y se escala a revisión humana, en lugar de escribir en Airtable un valor que el campo de selección rechazaría.
Solicitante	Nodo de búsqueda con	El ticket se crea con el correo en texto y sin

Punto de fallo	Configuración	Comportamiento
desconocido	*Always Output Data*	vínculo al maestro. Se notifica para que se agregue a Solicitantes. No es motivo para detener nada.
Activo no identificado	Ruta alterna de asignación	Se asigna por especialidad según la categoría. El ticket no queda huérfano.
Gmail rechaza el envío	On Error: salida de error	El ticket queda en estado Error con la respuesta pendiente, y aparece en el resumen diario.
Airtable no disponible	Reintentos, luego Error Workflow	La ejecución falla de forma controlada. El correo sigue sin leer en el buzón, por lo que el siguiente sondeo lo recoge.
Cualquier fallo no capturado	Error Workflow independiente	Registra rama, nodo, tipo y mensaje en el log y alerta por Telegram.

4.3 Control de bucles y de tipos

Riesgo	Control
El flujo se dispara con sus propias respuestas	El Gmail Trigger descarta los mensajes cuyo remitente es la propia cuenta (-from:me). Sin este filtro, cada acuse de recibo generaría un ticket nuevo, que generaría otro acuse: es el bucle infinito clásico de este diseño.
Varios tickets para la misma conversación	Antes de clasificar, se busca si el threadId ya tiene ticket. Si existe, la ejecución termina sin llamar al modelo y sin crear nada.
Reenvío del mismo correo	El messageId queda almacenado, lo que permite detectar duplicados exactos.
Comparaciones entre tipos distintos	SLA horas y horas de respuesta son numéricos y se comparan entre sí. La criticidad se traduce a un nivel numérico (1 a 4) antes de compararla con la del activo, en lugar de comparar etiquetas de texto.
Valores inventados en campos de selección	Categoría y criticidad se validan contra listas cerradas en el nodo Code antes de escribir en Airtable.

4.4 Puntos de intervención humana

El principio es que ninguna comunicación con contenido generado por IA llega a un solicitante sin que una persona la haya visto.

Punto	Canal	Qué se decide
Aprobación de la respuesta a tickets de criticidad alta o crítica	Telegram, botones Aprobar y Rechazar	El flujo se detiene antes del envío. Si se aprueba, sale el correo y se notifica al técnico; si se rechaza, el ticket pasa a

Punto	Canal	Qué se decide
		Gestión manual y no se envía nada.
Tickets con confianza baja	Telegram	Cuando el modelo no logra clasificar con seguridad, no hay respuesta automática: se notifica para lectura directa.
Incumplimientos de SLA	Telegram, resumen diario	El sistema no reasigna ni escala por su cuenta: entrega la lista de tickets vencidos para que una persona decida.
Fallos de datos y de API	Telegram, alerta inmediata	Se avisa en el momento, con el detalle en el log.
Criticidad baja y media	Automático, con salvaguarda	El acuse sale solo porque es una plantilla fija sin contenido generado. La criticidad que define esta ruta ya pasó por las reglas de negocio, no es la que propuso el modelo.

5. Dashboard de control

El panel no es el enlace a la base: es una vista construida sobre datos que el propio flujo genera. Cada tarde, la rama 2 calcula los indicadores del día y escribe un registro en la tabla KPIs diarios. Eso convierte el dashboard en un histórico acumulado y permite ver tendencias: si el cumplimiento de SLA se deteriora, si una categoría concentra los tickets, si la IA está clasificando peor que la semana pasada.

5.1 Indicadores

Indicador	Cálculo
Tasa de aprobación	`Aprobados / (Aprobados + Rechazados)`
Volumen de salida	Tickets con respuesta efectivamente enviada al solicitante
Tasa de error	`Errores / Recibidos`

Indicador	Cálculo	Qué revela
Cumplimiento de SLA	Cumplen SLA / (Cumplen + Incumplen)	El indicador principal de la mesa de ayuda.
Tiempo medio de primera respuesta	Promedio de Horas de respuesta	Mide el efecto real de la automatización: en las rutas con plantilla baja a minutos.
Tickets por categoría	Conteo agrupado	Señala dónde invertir en capacitación o en mantenimiento preventivo.
Proporción de	Criticidad alta /	Si sube de forma sostenida, hay un

Indicador	Cálculo	Qué revela
criticidad alta	Recibidos	problema de fondo en planta.
Solicitudes incompletas	Requieren info / Recibidos	Mide la calidad con que la gente reporta. Justifica mejorar el formato de solicitud.

5.2 Vistas publicadas

Vista	Tabla	Contenido
Panel de KPIs	KPIs diarios	Histórico por fecha con las tasas calculadas y gráfico de evolución. Es la vista principal del panel.
Tickets por estado y criticidad	Tickets	Agrupada por estado, con la criticidad como color. Muestra la carga viva de la mesa.
Monitor de errores	Log de errores	Filtrada por no resueltos, ordenada por fecha descendente.

Enlaces públicos:

Panel de KPIs: https://airtable.com/appJRP55y895G8kcz/shrNFXkAp3Hvtu2QG/tbl2X3ilcsPqLWOl4

Tickets por estado:
https://airtable.com/appJRP55y895G8kcz/shrNfYcrT7fdo2NDt/tblpOC9I8q3blVoaE

Monitor de errores:
https://airtable.com/appJRP55y895G8kcz/shri9fNSIE9suSIwZ/tblRtsHGnTt2SWMx1

Base en modo lectura:
https://airtable.com/appJRP55y895G8kcz/tblziuWDUJlOgCfqG/viw9ITVe3iZcL5SYT

6. Salida multicanal

Canal	Destinatario	Mensajes y estándar técnico aplicado
Gmail	Solicitante	Acuse de recibo, solicitud de datos faltantes y respuesta a tickets críticos. Los tres viajan en el hilo original: al recibir el correo se guarda el threadId y las respuestas se envían sobre el messageId, de modo que el solicitante ve una sola conversación.
Telegram	Responsable de la mesa	Aprobación de borradores con botones, alertas de error y resumen diario con los incumplimientos de SLA.
Telegram	Técnico asignado	Notificación de asignación al chat del técnico, tomado del maestro. Incluye ticket, criticidad, resumen y SLA.

7. Pruebas realizadas

El flujo se ejecutó ocho veces cubriendo el camino feliz y el infeliz. Los cuatro últimos casos validan los filtros y las rutas de error.

#	Caso de prueba	Resultado
1	Criticidad baja: instalación de software en equipo nuevo	Clasificado como Software y ERP, SLA 24 h, acuse de plantilla y aviso al técnico de TI. Sin intervención humana
2	Criticidad crítica: envasadora con parada de línea	La regla eleva la criticidad a Crítica, SLA 2 h, borrador generado y correo enviado solo tras aprobar en Telegram
3	Rechazo del borrador en Telegram	No se envía ningún correo al solicitante; el ticket queda en Gestión manual con decisión Rechazado
4	Descripción incompleta	Información suficiente en falso, respuesta de plantilla pidiendo los datos faltantes, estado Requiere info
5	Respuesta del solicitante en un hilo existente	El flujo detecta el threadId, no crea un segundo ticket y no llama al modelo
6	Solicitante fuera del maestro	El ticket se crea con el correo en texto y sin vínculo; el flujo continúa con normalidad
7	API de IA con credencial inválida	El ticket se crea en estado Error sin clasificar, se registra en el log y llega la alerta por Telegram
8	Urgencia exagerada por una impresora	El modelo propone criticidad alta por el tono; las reglas la bajan porque el activo no es crítico y no hay parada de línea